

OSSP-PRACTICAL-WEEK12

TASK1: Implement the Producer–Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        usleep(50000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
```

```

GNU nano 7.2
for (int i = 1; i <= ITEMS; i++)
{
    sem_wait(&full);
    pthread_mutex_lock(&mutex);

    int item = buffer[out];
    printf("Consumed: %d\n", item);
    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);

    usleep(80000);
}

return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    printf("Practical Week 12 - Task 1\n");
    printf("-----\n");
    printf("Buffer size: %d\n", BUFFER_SIZE);
    printf("Total items: %d\n\n", ITEMS);

    clock_t start = clock();

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_t end = clock();

    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nAll items processed successfully.\n");
    printf("Execution time: %.4f seconds\n", time_taken);

    if (time_taken > 0)
        printf("Throughput: %.2f items/second\n",
            ITEMS / time_taken);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Output:

```

root@LAPTOP-1C0JKKG6:~# nano practical_week12_task1.c
root@LAPTOP-1C0JKKG6:~# gcc practical_week12_task1.c -o practical_week12_task1 -pthread
root@LAPTOP-1C0JKKG6:~# ./practical_week12_task1
Practical Week 12 - Task 1
-----
Buffer size: 5
Total items: 20

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Consumed: 4
Produced: 6
Produced: 7
Consumed: 5
Produced: 8
Consumed: 6
Produced: 9
Produced: 10
Consumed: 7
Produced: 11
Produced: 12
Consumed: 8
Produced: 13
Consumed: 9
Produced: 14
Consumed: 10
Produced: 15
Consumed: 11
Produced: 16
Consumed: 12
Produced: 17
Consumed: 13
Produced: 18
Consumed: 14
Produced: 19
Consumed: 15
Produced: 20
Consumed: 16
Consumed: 17
Consumed: 18
Consumed: 19
Consumed: 20

All items processed successfully.
Execution time: 0.0082 seconds
Throughput: 2443.79 items/second
root@LAPTOP-1C0JKKG6:~# |

```

Observation:

Task2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Source code:

GNU nano 7.2

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    printf("Thread 1 waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1 acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    sleep(1);

    printf("Thread 2 waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2 acquired Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t thread1;
```

```

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("Deadlock Demonstration\n");
    printf("-----\n");

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

Output:

```

root@LAPTOP-1C0JKKG6:~# nano practical_week12_task2_deadlock.c
root@LAPTOP-1C0JKKG6:~# gcc practical_week12_task2_deadlock.c -o practical_week12_
root@LAPTOP-1C0JKKG6:~# ./practical_week12_task2_deadlock
Deadlock Demonstration
-----
Thread 1 locked Resource 1
Thread 2 locked Resource 2
Thread 1 waiting for Resource 2
Thread 2 waiting for Resource 1

```

Observation: I learned how to implement the Producer–Consumer problem using POSIX threads, counting semaphores, and mutex locks. I understood how semaphores manage empty and full buffer slots and how synchronization ensures correct data exchange between the producer and consumer. I also learned how buffer size can be changed to study execution time and throughput.

Task2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Source code:

GNU nano 7.2

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    printf("Thread 1 completed successfully.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    // Always acquire Resource 1 before Resource 2
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    printf("Thread 2 completed successfully.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
}
```

```

pthread_mutex_unlock(&resource1);

return NULL;
}

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("Deadlock Prevention Using Resource Ordering\n");
    printf("-----\n");

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\nBoth threads completed without deadlock.\n");

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano practical_week12_task2_prevention.c
root@LAPTOP-1COJKKG6:~# gcc practical_week12_task2_prevention.c -o practical_w
root@LAPTOP-1COJKKG6:~# ./practical_week12_task2_prevention
Deadlock Prevention Using Resource Ordering
-----
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 1 completed successfully.
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 2 completed successfully.

Both threads completed without deadlock.
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how deadlocks occur when multiple threads compete for shared resources. I understood the four necessary conditions for deadlock: mutual exclusion, hold and wait, no preemption, and circular wait. I also learned how resource ordering prevents circular wait by ensuring that all threads acquire resources in the same order.